

Лабораторная работа №3: Тестирование программ на Python и методология TDD

1. Теоретические сведения

1.1. Введение в тестирование программного обеспечения

Тестирование программного обеспечения — это процесс проверки и оценки программного продукта с целью выявления ошибок, дефектов и соответствия требованиям. В контексте разработки на Python, тестирование играет критически важную роль, обеспечивая надежность, стабильность и предсказуемость работы приложений. Тесты служат не только для обнаружения багов, но и как форма живой документации, демонстрирующей ожидаемое поведение кода [1].

Основные цели тестирования:

- **Выявление дефектов:** Обнаружение ошибок и несоответствий в коде.
- **Проверка соответствия требованиям:** Убедиться, что программа выполняет заявленные функции.
- **Повышение качества кода:** Тестирование способствует написанию более чистого, модульного и поддерживаемого кода.
- **Регрессионное тестирование:** Гарантия того, что новые изменения не нарушили существующую функциональность.
- **Уверенность в изменениях:** Разработчики могут вносить изменения в код, будучи уверенными, что тесты предупредят о возможных проблемах.

Виды тестирования (в контексте данной лабораторной работы):

- **Модульное (Unit Testing):** Тестирование отдельных, наименьших частей программы (функций, методов, классов) в изоляции. Цель — убедиться, что каждый компонент работает правильно.
- **Интеграционное (Integration Testing):** Проверка взаимодействия между различными модулями или компонентами системы. Цель — выявить проблемы, возникающие при их совместной работе.
- **Функциональное (Functional Testing):** Проверка функциональности программы с точки зрения пользователя, без учета внутренней структуры.

1.2. Модульное тестирование с `unittest`

`unittest` — это встроенный в Python фреймворк для модульного тестирования, вдохновленный JUnit. Он предоставляет набор инструментов для создания и запуска тестов, а также для проверки результатов.

Основные компоненты `unittest`:

- **`TestCase`**: Класс, который содержит набор тестовых методов. Каждый тестовый метод должен начинаться с префикса `test_`.
- **`assert` методы**: Методы, используемые внутри тестовых методов для проверки условий. Например, `assertEqual()`, `assertTrue()`, `assertRaises()`.
- **`setUp()` и `tearDown()`**: Методы, которые выполняются до и после каждого тестового метода соответственно. Используются для инициализации и очистки ресурсов.
- **`setUpClass()` и `tearDownClass()`**: Методы, которые выполняются один раз до и после всех тестовых методов в классе `TestCase`.

Пример использования `unittest`:

Предположим, у нас есть простая функция для сложения чисел:

```
# my_math.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

Теперь напомним тесты для этой функции с использованием `unittest`:

```
# test_my_math.py
import unittest
from my_math import add, subtract

class TestMyMath(unittest.TestCase):

    def setUp(self): # Выполняется перед каждым тестовым методом
        print("\nНачало теста...")
        self.a = 10
        self.b = 5

    def tearDown(self): # Выполняется после каждого тестового метода
        print("Завершение теста.")
```

```
del self.a
del self.b

def test_add_positive_numbers(self):
    """Тестирование сложения положительных чисел."""
    result = add(self.a, self.b)
    self.assertEqual(result, 15, "Сложение положительных чисел работает некорректно")

def test_add_negative_numbers(self):
    """Тестирование сложения отрицательных чисел."""
    result = add(-5, -3)
    self.assertEqual(result, -8)

def test_subtract_numbers(self):
    """Тестирование вычитания чисел."""
    result = subtract(self.a, self.b)
    self.assertEqual(result, 5)

def test_subtract_with_negative_result(self):
    """Тестирование вычитания с отрицательным результатом."""
    result = subtract(self.b, self.a)
    self.assertEqual(result, -5)

def test_add_with_zero(self):
    """Тестирование сложения с нулем."""
    result = add(self.a, 0)
    self.assertEqual(result, 10)

def test_subtract_with_zero(self):
    """Тестирование вычитания с нулем."""
    result = subtract(self.a, 0)
    self.assertEqual(result, 10)

def test_add_type_error(self):
    """Тестирование ошибки типа при сложении."""
    with self.assertRaises(TypeError):
        add(self.a, "b")

if __name__ == '__main__':
    unittest.main()
```

Запуск тестов `unittest`:

Для запуска тестов можно использовать команду:

```
python -m unittest test_my_math.py
```

Или просто запустить файл `test_my_math.py` как обычный скрипт Python, если в нем есть `if __name__ == '__main__': unittest.main()`.

1.3. Фреймворк `pytest`

`pytest` — это мощный и популярный фреймворк для тестирования в Python, который значительно упрощает написание тестов по сравнению с `unittest`. Он известен своей простотой, гибкостью и обширной экосистемой плагинов [2].

Основные преимущества `pytest`:

- **Простота написания тестов:** Не требует наследования от `TestCase` и использования специальных `assert` методов. Можно использовать стандартный оператор `assert`.
- **Автоматическое обнаружение тестов:** `pytest` автоматически находит тестовые файлы (начинающиеся с `test_` или заканчивающиеся на `_test.py`) и тестовые функции/методы.
- **Фикстуры (Fixtures):** Мощный механизм для подготовки тестового окружения и очистки ресурсов, который гораздо гибче, чем `setUp/tearDown` в `unittest`.
- **Параметризация тестов:** Возможность запускать один и тот же тест с разными наборами входных данных.
- **Подробные отчеты:** `pytest` предоставляет информативные отчеты о прохождении тестов.

Установка `pytest`:

```
pip install pytest
```

Пример использования `pytest`:

Используем те же функции `add` и `subtract` из `my_math.py`.

```
# test_my_math_pytest.py
from my_math import add, subtract
import pytest

def test_add_positive_numbers():
    assert add(10, 5) == 15

def test_add_negative_numbers():
    assert add(-5, -3) == -8

def test_subtract_numbers():
    assert subtract(10, 5) == 5
```

```
def test_subtract_with_negative_result():
    assert subtract(5, 10) == -5

def test_add_with_zero():
    assert add(10, 0) == 10

def test_subtract_with_zero():
    assert subtract(10, 0) == 10

def test_add_type_error():
    with pytest.raises(TypeError):
        add(10, "b")

# Пример параметризованного теста
@pytest.mark.parametrize("a, b, expected", [
    (1, 2, 3),
    (-1, -2, -3),
    (0, 0, 0),
    (100, -50, 50),
])
def test_add_parametrized(a, b, expected):
    assert add(a, b) == expected

# Пример использования фикстуры
@pytest.fixture
def setup_numbers():
    """Фикстура, предоставляющая числа для тестов."""
    print("\nПодготовка чисел для теста...")
    yield 10, 5 # Возвращает кортеж (a, b)
    print("Очистка после теста.")

def test_add_with_fixture(setup_numbers):
    a, b = setup_numbers
    assert add(a, b) == 15

def test_subtract_with_fixture(setup_numbers):
    a, b = setup_numbers
    assert subtract(a, b) == 5
```

Запуск тестов `pytest`:

Для запуска тестов достаточно перейти в директорию с тестовыми файлами и выполнить команду:

```
pytest
```

Для более подробного вывода можно использовать:

```
pytest -v
```

Для запуска тестов из определенного файла:

```
pytest test_my_math_pytest.py
```

1.4. Продвинутые возможности `pytest`

1.4.1. Фикстуры (`Fixtures`)

Фикстуры в `pytest` — это функции, которые подготавливают тестовое окружение (например, создают временные файлы, настраивают базу данных, инициализируют объекты) и могут быть использованы в тестовых функциях или других фикстурах. Они обеспечивают повторное использование кода настройки и очистки.

Область видимости фикстур (`scope`):

- `function` (по умолчанию): Фикстура выполняется один раз для каждого тестового вызова.
- `class`: Фикстура выполняется один раз для каждого тестового класса.
- `module`: Фикстура выполняется один раз для каждого модуля.
- `session`: Фикстура выполняется один раз для всей тестовой сессии.

```
# conftest.py (файл для общих фикстур)
import pytest

@pytest.fixture(scope="module")
def db_connection():
    """Фикстура для имитации подключения к базе данных."""
    print("\n[SETUP] Открытие соединения с БД")
    conn = {"status": "connected"} # Имитация объекта соединения
    yield conn
    print("[TEARDOWN] Закрытие соединения с БД")

@pytest.fixture(scope="function")
def user_data():
    """Фикстура, предоставляющая тестовые данные пользователя."""
    print("\n[SETUP] Подготовка данных пользователя")
    data = {"name": "Test User", "email": "test@example.com"}
    yield data
    print("[TEARDOWN] Очистка данных пользователя")
```

```
# test_database.py
import pytest

def test_fetch_user(db_connection, user_data):
    """Тест получения пользователя из БД."""
    print(f"Тест: Получение пользователя {user_data['name']} через {db_connection}")
    assert db_connection["status"] == "connected"
    assert user_data["name"] == "Test User"

def test_add_user(db_connection):
    """Тест добавления пользователя в БД."""
    print(f"Тест: Добавление пользователя через {db_connection['status']} БД")
    assert db_connection["status"] == "connected"
```

При запуске `pytest` фикстуры из `conftest.py` будут автоматически обнаружены и доступны для всех тестов в этой директории и ее поддиректориях.

1.4.2. Параметризация тестов (`parametrize`)

Декоратор `@pytest.mark.parametrize` позволяет запускать один и тот же тестовый код несколько раз с разными наборами входных данных. Это значительно сокращает дублирование кода и делает тесты более читаемыми и поддерживаемыми.

```
# test_calculator_parametrized.py
import pytest

def divide(a, b):
    if b == 0:
        raise ValueError("Деление на ноль невозможно")
    return a / b

@pytest.mark.parametrize("num1, num2, expected_result", [
    (10, 2, 5.0),
    (10, 5, 2.0),
    (-10, 2, -5.0),
    (5, 2, 2.5),
])

def test_divide_valid_inputs(num1, num2, expected_result):
    assert divide(num1, num2) == expected_result

@pytest.mark.parametrize("num1, num2", [
    (10, 0),
    (0, 0),
])
```

```
def test_divide_by_zero_raises_error(num1, num2):  
    with pytest.raises(ValueError, match="Деление на ноль невозможно"):  
        divide(num1, num2)
```

1.5. Изоляция тестов и Mock-объекты

При модульном тестировании крайне важно тестировать каждый компонент в изоляции. Однако реальные приложения часто зависят от внешних ресурсов, таких как базы данных, сетевые API, файловая система или системное время. Прямое взаимодействие с этими зависимостями во время тестов может привести к следующим проблемам:

- **Медленные тесты:** Сетевые запросы или операции с базой данных занимают много времени.
- **Нестабильные тесты:** Внешние сервисы могут быть недоступны или возвращать разные данные, что делает тесты непредсказуемыми.
- **Сложность настройки:** Для каждого теста может потребоваться сложная настройка окружения (например, заполнение базы данных тестовыми данными).
- **Побочные эффекты:** Тесты могут изменять состояние внешних систем, влияя на другие тесты или даже на реальные данные.

Для решения этих проблем используются **Mock-объекты** (заглушки, имитации). Mock-объект — это поддельный объект, который имитирует поведение реального объекта, но находится под полным контролем теста. Это позволяет изолировать тестируемый код от его зависимостей.

В Python для создания Mock-объектов используется модуль `unittest.mock` (который также хорошо интегрируется с `pytest`).

Основные классы и функции `unittest.mock`:

- **Mock**: Базовый класс для создания Mock-объектов. Позволяет задавать возвращаемые значения (`return_value`), имитировать исключения (`side_effect`), отслеживать вызовы (`called`, `call_count`, `assert_called_once_with`).
- **MagicMock**: Расширение `Mock`, которое имитирует все магические методы Python (например, `__len__`, `__str__`, `__getitem__`), что делает его более универсальным для подмены различных типов объектов.
- **patch**: Декоратор или контекстный менеджер, который временно заменяет объект другим (обычно Mock-объектом) на время выполнения теста. Это самый распространенный способ использования Mock-объектов, так как он позволяет подменять зависимости там, где они импортируются или используются.

Пример использования `Mock` и `patch`:

Предположим, у нас есть функция, которая получает данные о пользователе из внешнего API:

```
# user_service.py
import requests

class UserService:
    def __init__(self, api_base_url):
        self.api_base_url = api_base_url

    def get_user_data(self, user_id):
        url = f"{self.api_base_url}/users/{user_id}"
        try:
            response = requests.get(url, timeout=5)
            response.raise_for_status() # Вызывает исключение для ошибок HTTP
            return response.json()
        except requests.exceptions.RequestException as e:
            print(f"Ошибка при запросе данных пользователя: {e}")
            return None

    def get_user_email(self, user_id):
        user_data = self.get_user_data(user_id)
        if user_data:
            return user_data.get("email")
        return None
```

Чтобы протестировать `UserService` без выполнения реальных сетевых запросов, мы можем использовать `patch` для подмены `requests.get`:

```
# test_user_service.py
import unittest
from unittest.mock import patch, Mock
from user_service import UserService

class TestUserService(unittest.TestCase):

    def setUp(self):
        self.service = UserService("http://api.example.com")

    @patch("user_service.requests") # Подменяем модуль requests внутри user_serv
    def test_get_user_data_success(self, mock_requests):
        # Создаем Mock-объект для ответа
        mock_response = Mock()
        mock_response.status_code = 200
```

```

mock_response.json.return_value = {"id": 1, "name": "Test User", "email": "test@example.com"}
mock_response.raise_for_status.return_value = None # raise_for_status не вызывает исключение

# Настраиваем mock_requests.get так, чтобы он возвращал наш mock_response
mock_requests.get.return_value = mock_response

user_data = self.service.get_user_data(1)

# Проверяем, что requests.get был вызван с правильными аргументами
mock_requests.get.assert_called_once_with("http://api.example.com/users/1")
self.assertEqual(user_data["name"], "Test User")
self.assertEqual(user_data["email"], "test@example.com")

@patch("user_service.requests")
def test_get_user_data_http_error(self, mock_requests):
    mock_response = Mock()
    mock_response.status_code = 404
    # Имитируем вызов raise_for_status, который должен вызвать исключение
    mock_response.raise_for_status.side_effect = requests.exceptions.HTTPError
    mock_requests.get.return_value = mock_response

    user_data = self.service.get_user_data(2)

    mock_requests.get.assert_called_once_with("http://api.example.com/users/2")
    self.assertIsNone(user_data)

@patch("user_service.UserService.get_user_data") # Подменяем метод get_user_data
def test_get_user_email_success(self, mock_get_user_data):
    mock_get_user_data.return_value = {"id": 3, "name": "Another User", "email": "another@example.com"}

    email = self.service.get_user_email(3)

    mock_get_user_data.assert_called_once_with(3)
    self.assertEqual(email, "another@example.com")

@patch("user_service.UserService.get_user_data")
def test_get_user_email_no_data(self, mock_get_user_data):
    mock_get_user_data.return_value = None

    email = self.service.get_user_email(4)

    mock_get_user_data.assert_called_once_with(4)
    self.assertIsNone(email)

```

Ключевые моменты при использовании `patch`:

- **Путь для `patch`**: Всегда указывайте путь к объекту, который вы хотите подменить, *там, где он используется*, а не там, где он определен. Например, если `requests` импортируется в `user_service.py` как `import requests`, то путь для `patch` будет `user_service.requests`.
- **Возвращаемое значение (`return_value`)**: Используется для определения того, что Mock-объект должен вернуть при вызове.
- **Побочный эффект (`side_effect`)**: Позволяет Mock-объекту вызывать исключения или возвращать разные значения при последовательных вызовах.
- **Проверка вызовов (`assert_called_once_with`, `call_count` и т.д.)**: Позволяет убедиться, что Mock-объект был вызван ожидаемым образом.

Использование Mock-объектов позволяет создавать быстрые, надежные и изолированные модульные тесты, которые не зависят от состояния внешних систем и не требуют сложной настройки окружения.

1.6. Методология Test-Driven Development (TDD)

Test-Driven Development (TDD), или разработка через тестирование, — это методология разработки программного обеспечения, при которой тесты пишутся *до* написания рабочего кода. Это не просто техника тестирования, а полноценный подход к разработке, который влияет на дизайн кода, его архитектуру и качество [4].

Основная идея TDD заключается в следующем: вместо того чтобы сначала писать весь функционал, а затем проверять его тестами, разработчик сначала определяет, что именно должен делать код, описывает это в виде теста, а затем пишет минимальный код, который заставит этот тест пройти.

Цикл TDD состоит из трех основных шагов, часто называемых «Красный-Зеленый-Рефакторинг» (Red-Green-Refactor):

1. **Красный (Red)**: Напишите небольшой, проваливающийся тест. Этот тест должен описывать новую, еще не реализованную функциональность или исправлять известный баг. Важно, чтобы тест провалился, так как это подтверждает, что:
 - Тест действительно проверяет то, что вы хотите.
 - Функциональность еще не реализована.
 - Тестовое окружение настроено правильно.
2. **Зеленый (Green)**: Напишите *минимальный* объем кода, который заставит только что написанный тест пройти. На этом этапе не стоит беспокоиться о чистоте кода, его оптимальности или архитектуре. Главная цель —

заставить тест пройти как можно быстрее. Если есть другие проваливающиеся тесты, их пока игнорируют.

3. **Рефакторинг (Refactor):** После того как тест стал зеленым (прошел), можно безопасно улучшать код. На этом этапе можно переименовывать переменные, изменять структуру функций, удалять дублирование, улучшать читаемость и оптимизировать производительность, не опасаясь сломать что-либо, поскольку все тесты (включая только что написанный) должны оставаться зелеными. Если в процессе рефакторинга какой-либо тест начинает падать, это означает, что вы внесли ошибку, и ее нужно немедленно исправить.

После завершения цикла «Красный-Зеленый-Рефакторинг» процесс повторяется для следующей порции функциональности.

Преимущества TDD:

- **Улучшенный дизайн кода:** TDD заставляет разработчика думать об интерфейсе и использовании кода до его реализации, что часто приводит к более чистому, модульному и легко тестируемому дизайну.
- **Меньше багов:** Постоянное написание и запуск тестов помогает выявлять ошибки на ранних стадиях разработки.
- **Живая документация:** Тесты служат актуальной и исполняемой документацией, показывающей, как должен работать код.
- **Уверенность в изменениях:** Наличие полного набора тестов дает уверенность при рефакторинге или добавлении нового функционала.
- **Ускорение разработки в долгосрочной перспективе:** Хотя TDD может показаться медленным на начальном этапе, в долгосрочной перспективе оно сокращает время на отладку и исправление ошибок.

Пример TDD-цикла: Разработка функции для форматирования телефонного номера

Предположим, нам нужно разработать функцию

`format_phone_number(number_string)`, которая принимает строку с телефонным номером (возможно, с лишними символами) и возвращает его в стандартном формате `+X (XXX) XXX-XX-XX`.

Шаг 1: Красный (Red) - Написание проваливающегося теста

Сначала создадим файл `phone_formatter.py` (пока пустой) и `test_phone_formatter.py`.

```
# test_phone_formatter.py
import pytest
from phone_formatter import format_phone_number # Эта строка пока вызовет ошибку
```

```
def test_format_simple_number():
    # Ожидаем, что функция вернет отформатированный номер
    assert format_phone_number("79001234567") == "+7 (900) 123-45-67"

def test_format_number_with_spaces_and_dashes():
    assert format_phone_number("+7 900 123-45-67") == "+7 (900) 123-45-67"

def test_format_number_with_parentheses():
    assert format_phone_number("8 (900) 123 45 67") == "+7 (900) 123-45-67"

def test_format_short_number_raises_error():
    with pytest.raises(ValueError, match="Некорректная длина номера"):
        format_phone_number("123")

def test_format_non_digit_characters_raises_error():
    with pytest.raises(ValueError, match="Номер содержит недопустимые символы"):
        format_phone_number("7900abc4567")
```

При запуске `pytest` мы получим ошибки, так как `phone_formatter.py` пуст, и функция `format_phone_number` не существует. Это наш «Красный» этап.

Шаг 2: Зеленый (Green) - Написание минимального кода

Теперь напишем минимальный код в `phone_formatter.py`, чтобы тесты прошли. Мы не будем беспокоиться о красоте или универсальности, только о прохождении тестов.

```
# phone_formatter.py
import re

def format_phone_number(number_string):
    # Удаляем все, кроме цифр
    digits = re.sub(r'\D', '', number_string)

    # Если номер начинается с 8, заменяем на 7
    if digits.startswith('8'):
        digits = '7' + digits[1:]

    # Проверяем длину
    if len(digits) != 11:
        raise ValueError("Некорректная длина номера")

    # Проверяем на недопустимые символы (хотя re.sub уже должен был их удалить)
    if not digits.isdigit():
        raise ValueError("Номер содержит недопустимые символы")
```

```
# Форматируем
return f'''+{digits[0]} ({digits[1:4]}) {digits[4:7]}-{digits[7:9]}-{digits[9:]}
```

Запускаем `pytest`. Все тесты должны пройти. Это наш «Зеленый» этап.

Шаг 3: Рефакторинг (Refactor)

Теперь, когда тесты зеленые, мы можем улучшить код `format_phone_number`, не боясь что-либо сломать. Например, можно сделать регулярное выражение более строгим или улучшить обработку ошибок.

```
# phone_formatter.py (после рефакторинга)
import re

def format_phone_number(number_string):
    # Удаляем все нецифровые символы
    cleaned_number = re.sub(r'\D', '', number_string)

    # Проверяем, что остались только цифры
    if not cleaned_number.isdigit():
        raise ValueError("Номер содержит недопустимые символы")

    # Приводим к единому формату для России (начинается с 7)
    if cleaned_number.startswith('8'):
        cleaned_number = '7' + cleaned_number[1:]
    elif cleaned_number.startswith('9') and len(cleaned_number) == 10: # Если не
        cleaned_number = '7' + cleaned_number

    # Проверяем корректную длину для российского номера (11 цифр)
    if len(cleaned_number) != 11:
        raise ValueError("Некорректная длина номера")

    # Форматируем номер
    # Пример: +7 (900) 123-45-67
    return f'''+{cleaned_number[0]} ({cleaned_number[1:4]}) {cleaned_number[4:7]}-{cleaned_number[7:9]}-{cleaned_number[9:]}
```

Снова запускаем `pytest`. Все тесты должны по-прежнему быть зелеными. Если нет, значит, рефакторинг внес ошибку, и ее нужно исправить. Это наш «Рефакторинг» этап.

Этот цикл повторяется для каждой новой функциональности, обеспечивая постоянное качество и надежность кода.

✓ 2. Практические задания

Задание №1: Тестирование функций парсинга данных

Возьмите парсер, разработанный в Лабораторной работе №1 (задание №1).

Требования:

- Создайте отдельный модуль (например, `test_parser`) для тестов.
- Используйте `pytest` для написания юнит-тестов для функций вашего парсера.
- Проверьте, что функции корректно извлекают данные (например, что возвращается список строк, что каждая строка не пуста).
- Используйте `unittest.mock.patch` для имитации HTTP-ответа от `requests.get` или `BeautifulSoup` при загрузке страницы.
- Убедитесь, что ваш парсер корректно обрабатывает случаи, когда данные отсутствуют на странице или имеют неожиданный формат.

Задание №2: Mock-тестирование API-клиента для погодного сервиса

В Лабораторной работе №1 в рамках задачи №2 вы работали с внешними API (с погодным сервисом). Используя наработки из прошлой работы создайте (или используйте существующий) простой клиент для погодного API, который принимает название города и возвращает текущую температуру и описание погоды.

Требования:

- Напишите тесты для вашего API-клиента, используя `pytest`.
- Используйте `unittest.mock.patch` для подмены `requests.get` (или аналогичной функции) и имитации ответов от погодного API.
- Убедитесь, что ваш клиент корректно обрабатывает ситуации ответа API, возвращая `None` или генерируя специфические исключения.

Задание №3: Контрольные вопросы

1. Объясните разницу между модульным, интеграционным и функциональным тестированием.
2. В чем заключаются основные преимущества использования тестов в процессе разработки?

3. Какие основные отличия `pytest` от `unittest`? Почему `pytest` считается более предпочтительным в современной разработке?
4. Что такое фикстуры в `pytest` и для чего они используются?
5. Что такое Test-Driven Development (TDD)? Опишите его основные этапы.
6. Что такое рефакторинг в контексте TDD и почему он важен?
7. Что такое Mock-объекты и зачем они нужны при тестировании?
8. Какие проблемы могут возникнуть при тестировании функций, зависящих от внешних ресурсов, без использования Mock-объектов?
9. Как TDD способствует улучшению дизайна кода?